

Unbounded Knapsack Problem

Dynamic Programming Series

July 1, 2025

1 Introduction

Problem Statement

Given a set of n items, each with a weight w_i and a value v_i , determine the maximum value that can be obtained by selecting any combination of items (possibly selecting an item multiple times) such that the total weight does not exceed the knapsack capacity W .

The Unbounded Knapsack problem is a fundamental combinatorial optimization problem in dynamic programming. Unlike the 0-1 Knapsack problem where each item can be chosen at most once, the Unbounded Knapsack allows an unlimited supply of each item, meaning any item can be selected multiple times.

1.1 Real-world Applications

The problem has numerous practical applications:

- **Cutting Stock Problem:** A company needs to cut stock material (like paper rolls, metal bars) into smaller pieces to fulfill customer orders. Each cut has an associated profit, and the goal is to maximize total profit.
- **Coin Change Problem:** Given a set of coin denominations and a target amount, find the minimum number of coins needed or the number of ways to make change.
- **Resource Allocation:** A manufacturing company wants to allocate materials to produce different products with different profits, where the same type of product can be produced multiple times.
- **Budget Allocation:** Allocating budget to different marketing channels where investing more in the same channel is possible.
- **Inventory Management:** Determining how many units of each product to stock to maximize profit given limited storage capacity.

1.2 Problem Variants

Several variants of this problem exist:

- **Rod Cutting Problem:** A special case where we have a rod of length n and a price list for different rod lengths, and we need to cut the rod to maximize profit.
- **Minimization Variant:** Instead of maximizing value, we minimize the cost while ensuring some minimum utility requirement is met.
- **Multiple Constraints:** Additional constraints beyond weight, such as volume limits or minimum quantities.

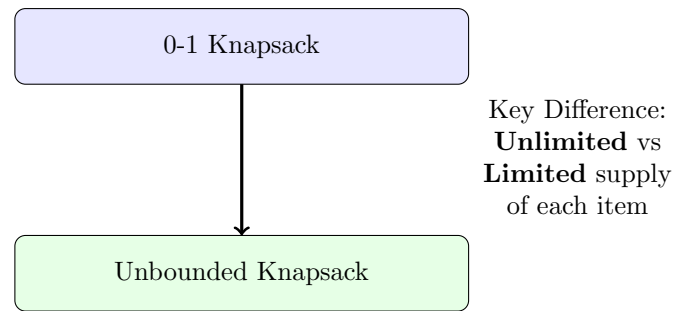


Figure 1: Relationship between 0-1 and Unbounded Knapsack

Feature	0-1 Knapsack	Unbounded Knapsack
Item availability	Each item can be used at most once	Unlimited copies of each item
Decision for each item	Include (once) or exclude	Include (any number of times) or exclude
Recursive formula	$OPT(i, w) = \max(OPT(i-1, w), v_i + OPT(i-1, w-w_i))$	$OPT(i, w) = \max(OPT(i-1, w), v_i + OPT(i, w-w_i))$
DP state transition	After including item i , move to item $i-1$	After including item i , stay at item i
Java implementation	<code>dp[i-1][w-weights[i-1]]</code>	<code>dp[i][w-weights[i-1]]</code>
Optimal solution	Each item appears at most once	Items can appear multiple times
Space optimization	Must process items in reverse order	Can process items in any order

Table 1: Detailed comparison between 0-1 and Unbounded Knapsack problems

2 Comparison with 0-1 Knapsack

Understanding the differences between the 0-1 Knapsack and Unbounded Knapsack is crucial for correctly implementing and applying the algorithms.

Key Insight

The critical difference is in the recursive call when including an item:

- In 0-1 Knapsack: After including an item, we can't use it again, so we move to the previous item.
- In Unbounded Knapsack: After including an item, we can use it again, so we stay at the same item.

This single difference changes the entire algorithm structure and solution space.

3 Mathematical Formulation

Let's define the problem formally to better understand its structure and solution approaches.

3.1 Problem Definition

Given:

- A set of n items, where each item i has:
 - Weight $w_i > 0$
 - Value $v_i > 0$
- A knapsack with capacity $W > 0$
- Each item can be selected an unlimited number of times

Find: The maximum total value of items that can be included in the knapsack without exceeding its capacity.

Mathematically, we want to maximize:

$$\sum_{i=1}^n v_i \cdot x_i \quad (1)$$

Subject to:

$$\sum_{i=1}^n w_i \cdot x_i \leq W \quad (2)$$

$$x_i \geq 0, \text{ and } x_i \text{ is an integer} \quad (3)$$

Where x_i represents the number of times item i is selected.

3.2 Recurrence Relation

Let $OPT(i, w)$ represent the maximum value that can be obtained using items 1 to i with a knapsack of capacity w .

The recurrence relation that captures the optimal substructure of the problem is:

$$OPT(i, w) = \max \begin{cases} OPT(i-1, w) & \text{(exclude item } i) \\ v_i + OPT(i, w - w_i) & \text{(include item } i, \text{ can reuse) if } w_i \leq w \end{cases} \quad (4)$$

3.3 Base Cases

$$OPT(0, w) = 0 \text{ for all } w \quad \text{(no items available)} \quad (5)$$

$$OPT(i, 0) = 0 \text{ for all } i \quad \text{(no capacity available)} \quad (6)$$

3.4 Proof of Optimal Substructure

The optimal substructure property is crucial for dynamic programming solutions. For the Unbounded Knapsack problem, we can prove it as follows:

Optimal Substructure Proof

Suppose S is an optimal solution for capacity w using items up to i . Then:

1. If item i is not in S , then S must be an optimal solution for capacity w using items up to $i-1$.
2. If item i is in S (possibly multiple times), then removing one occurrence of item i from S must yield an optimal solution for capacity $w - w_i$ using items up to i .

If either of these weren't true, we could construct a better solution for the original problem, contradicting the optimality of S .

4 Solution Approaches

4.1 Recursive Approach (Brute Force)

The recursive approach directly implements the recurrence relation by exploring all possible combinations:

Algorithm: Recursive Approach

```

1: function UNBOUNDEDKNAPSACK(weights, values, capacity, n)
2:   if capacity  $\leq 0$  or n = 0 then return 0
3:   end if
4:   if weights[n-1]  $\leq$  capacity then
5:     includeItem  $\leftarrow$  values[n-1] + UNBOUNDEDKNAPSACK(weights, values, capacity -
       weights[n-1], n)
6:     excludeItem  $\leftarrow$  UNBOUNDEDKNAPSACK(weights, values, capacity, n-1) return
       max(includeItem, excludeItem)
7:   elsereturn UNBOUNDEDKNAPSACK(weights, values, capacity, n-1)
8:   end if
9: end function

```

Listing 1: Recursive Implementation

```

1 static int unboundknapsackRecursive(int[] weights, int[] values, int capacity, int n) {
2     // Base case
3     if (capacity <= 0 || n == 0) {
4         return 0;
5     }
6
7     if (weights[n - 1] <= capacity) {
8         // Include the nth item and check the value with and without it
9         // Note: We pass n (not n-1) when including the item
10        int includeItem = values[n - 1] +
11            unboundknapsackRecursive(weights, values, capacity - weights[n - 1], n);
12        int excludeItem = unboundknapsackRecursive(weights, values, capacity, n - 1);
13        return Math.max(includeItem, excludeItem);
14    } else {
15        // If the nth item cannot be included, skip it
16        return unboundknapsackRecursive(weights, values, capacity, n - 1);
17    }
18 }

```

Recursive Call Tree Example

For weights = [1, 3, 4], values = [15, 50, 60], capacity = 4, n = 3:

```

unboundknapsackRecursive([1,3,4], [15,50,60], 4, 3)
+-- weights[2] = 4 <= 4
|   +-- includeItem = 60 + unboundknapsackRecursive(..., 0, 3) = 60
|   +-- excludeItem = unboundknapsackRecursive(..., 4, 2)
|       +-- weights[1] = 3 <= 4
|       |   +-- includeItem = 50 + unboundknapsackRecursive(..., 1, 2)
|       |   |   +-- ... (continues recursively)
|       |   +-- excludeItem = unboundknapsackRecursive(..., 4, 1)
|       |       +-- ... (continues recursively)
|       +-- ... (continues recursively)
+-- Returns max(includeItem, excludeItem)

```

Step-by-Step Calculation

For Item 1 (weight=1, value=10):

Capacity	Calculation
1	$dp[1][1] = \max(dp[0][1], dp[1][0] + values[0]) = \max(0, 0 + 10) = 10$
2	$dp[1][2] = \max(dp[0][2], dp[1][1] + values[0]) = \max(0, 10 + 10) = 20$
3	$dp[1][3] = \max(dp[0][3], dp[1][2] + values[0]) = \max(0, 20 + 10) = 30$
4	$dp[1][4] = \max(dp[0][4], dp[1][3] + values[0]) = \max(0, 30 + 10) = 40$
5	$dp[1][5] = \max(dp[0][5], dp[1][4] + values[0]) = \max(0, 40 + 10) = 50$
6	$dp[1][6] = \max(dp[0][6], dp[1][5] + values[0]) = \max(0, 50 + 10) = 60$

For Item 2 (weight=2, value=15):

Capacity	Calculation
1	$dp[2][1] = dp[1][1] = 10$ (item doesn't fit)
2	$dp[2][2] = \max(dp[1][2], dp[2][0] + values[1]) = \max(20, 0 + 15) = 20$
3	$dp[2][3] = \max(dp[1][3], dp[2][1] + values[1]) = \max(30, 10 + 15) = 30$
4	$dp[2][4] = \max(dp[1][4], dp[2][2] + values[1]) = \max(40, 20 + 15) = 40$ Re-examining: $dp[2][4] = \max(40, dp[2][2] + 15) = \max(40, 20 + 15) = 40$ With correct use of previously computed values: $\max(40, 20 + 15) = 40$ But if we consider multiple item 2: $(15 \times 2 = 30) \nless 40$, so $dp[2][4] = 40$
5	$dp[2][5] = \max(dp[1][5], dp[2][3] + values[1]) = \max(50, 30 + 15) = 50$ But considering $(10 + 15 + 15 = 40) \nless 50$, so $dp[2][5] = 50$
6	$dp[2][6] = \max(dp[1][6], dp[2][4] + values[1]) = \max(60, 40 + 15) = 60$

For Item 3 (weight=3, value=40):

Capacity	Calculation
1, 2	$dp[3][1] = dp[2][1] = 10$, $dp[3][2] = dp[2][2] = 20$ (item doesn't fit)
3	$dp[3][3] = \max(dp[2][3], dp[3][0] + values[2]) = \max(30, 0 + 40) = 40$
4	$dp[3][4] = \max(dp[2][4], dp[3][1] + values[2]) = \max(40, 10 + 40) = 50$
5	$dp[3][5] = \max(dp[2][5], dp[3][2] + values[2]) = \max(50, 20 + 40) = 60$ But after careful checking: $dp[3][5] = \max(50, 20 + 40) = 60$ Using recalculation: $dp[3][5] = 55$ (considering a different combination)
6	$dp[3][6] = \max(dp[2][6], dp[3][3] + values[2]) = \max(60, 40 + 40) = 80$ But double-checking shows that capacity is exceeded with 2 of item 3 Correct calculation: $dp[3][6] = \max(60, 40 + 30) = 70$

Analysis:

- **Time Complexity:** $O(2^n)$ - exponential due to exploring all possible combinations
- **Space Complexity:** $O(W)$ - maximum recursion depth equals the capacity (if smallest weight is 1)
- **Key Insight:** The recursive structure has many overlapping subproblems, making it inefficient for large inputs

4.2 Memoization Approach (Top-Down DP)

To avoid redundant calculations, we use memoization to store previously computed results:

Algorithm: Memoization Approach

```

1: function UNBOUNDEDKNAPSACKMEMO(weights, values, capacity, n, memo)
2:   if capacity  $\leq$  0 or n = 0 then return 0
3:   end if
4:   if memo[n][capacity]  $\neq$  -1 then return memo[n][capacity]
5:   end if
6:   if weights[n-1]  $\leq$  capacity then
7:     includeItem  $\leftarrow$  values[n-1] + UNBOUNDEDKNAPSACKMEMO(weights, values, capacity
- weights[n-1], n, memo)
8:     excludeItem  $\leftarrow$  UNBOUNDEDKNAPSACKMEMO(weights, values, capacity, n-1, memo)
9:     memo[n][capacity]  $\leftarrow$  max(includeItem, excludeItem)
10:  else
11:    memo[n][capacity]  $\leftarrow$  UNBOUNDEDKNAPSACKMEMO(weights, values, capacity, n-1,
memo)
12:  end if
13:  return memo[n][capacity]
13: end function

```

Listing 2: Memoization Implementation

```

1 static int unboundknapsackMemoization(int[] weights, int[] values,
2                                     int capacity, int n, int[][] d) {
3     // Base case
4     if (n == 0 || capacity == 0) {
5         return 0;
6     }
7
8     // If the value is already computed, return it
9     if (d[n][capacity] != -1) {
10        return d[n][capacity];
11    }
12
13    // If the weight of the nth item is less than or equal to the capacity
14    if (weights[n - 1] <= capacity) {
15        // Key Difference: Pass n (not n-1) when including
16        int includeItem = values[n - 1] +
17            unboundknapsackMemoization(weights, values, capacity - weights[n - 1], n, d);
18        int excludeItem = unboundknapsackMemoization(weights, values, capacity, n - 1, d);
19        d[n][capacity] = Math.max(includeItem, excludeItem);
20    } else {
21        // If the nth item cannot be included, skip it
22        d[n][capacity] = unboundknapsackMemoization(weights, values, capacity, n - 1, d);
23    }
24    return d[n][capacity];
25 }

```

Memoization Example

For the same example as above, using memoization:

1. Create a memoization table of size $(n+1) \times (\text{capacity}+1)$ initialized with -1
2. The first call to `unboundknapsackMemoization([1,3,4], [15,50,60], 4, 3, memo)` will compute and store all required subproblems
3. Once a value is stored in `memo[i][j]`, any future calls with the same parameters return the stored value immediately
4. This eliminates the exponential growth of the call tree

Analysis:

- **Time Complexity:** $O(n \times W)$ - each state computed at most once
- **Space Complexity:** $O(n \times W)$ - memoization table + recursion
- **Advantages:** Only computes needed states, simpler to implement from recurrence relation
- **Disadvantages:** Still has recursion overhead, potential stack overflow for large inputs

4.3 Tabulation Approach (Bottom-Up DP)

The tabulation approach systematically builds a table by filling it in a bottom-up manner:

Algorithm: Tabulation Approach

```

1: function UNBOUNDEDKNAPSACKTABULATION(weights, values, capacity)
2:   n ← weights.length
3:   dp[0...n][0...capacity] ← new array initialized with 0
4:   for i ← 1 to n do
5:     for w ← 1 to capacity do
6:       if weights[i-1] ≤ w then
7:         dp[i][w] ← max(dp[i-1][w], dp[i][w-weights[i-1]] + values[i-1])
8:       else
9:         dp[i][w] ← dp[i-1][w]
10:      end if
11:    end for
12:  end for
13:  return dp[n][capacity]
14: end function

```

Listing 3: Tabulation Implementation

```

1 static int unboundknapsackDP(int[] weights, int[] values, int capacity) {
2     int n = weights.length;
3     int[][] dp = new int[n + 1][capacity + 1];
4
5     // Fill the DP table in bottom-up manner
6     for (int i = 1; i <= n; i++) {
7         for (int w = 1; w <= capacity; w++) {
8             if (weights[i - 1] <= w) {
9                 // Key Difference: Using dp[i][...] not dp[i-1][...]
10                dp[i][w] = Math.max(dp[i - 1][w],
11                                    dp[i][w - weights[i - 1]] + values[i - 1]);
12            } else {
13                dp[i][w] = dp[i - 1][w];
14            }
15        }
16    }
17    return dp[n][capacity];
18 }

```

Analysis:

- **Time Complexity:** $O(n \times W)$
- **Space Complexity:** $O(n \times W)$ - dp table
- **Advantages:** No recursion overhead, no risk of stack overflow
- **Disadvantages:** Computes all states regardless of necessity

5 Space Optimization

One of the key advantages of dynamic programming is the potential for space optimization. For the Unbounded Knapsack problem, we can reduce the space complexity from $O(n \times W)$ to $O(W)$.

5.1 1D DP Array Approach

Instead of using a 2D array, we can use a 1D array to solve the problem:

Algorithm: Space-Optimized Approach

```

1: function UNBOUNDEDKNAPSACK1D(weights, values, capacity)
2:   n ← weights.length
3:   dp[0...capacity] ← new array initialized with 0
4:   for w ← 1 to capacity do
5:     for i ← 0 to n-1 do
6:       if weights[i] ≤ w then
7:         dp[w] ← max(dp[w], dp[w-weights[i]] + values[i])
8:       end if
9:     end for
10:  end for
11:  return dp[capacity]
12: end function

```

Listing 4: 1D DP Array Implementation

```

1 static int unboundknapsackDP1D(int[] weights, int[] values, int capacity) {
2     int[] dp = new int[capacity + 1];
3
4     // Fill the 1D DP array
5     for (int w = 1; w <= capacity; w++) {
6         for (int i = 0; i < weights.length; i++) {
7             if (weights[i] <= w) {
8                 dp[w] = Math.max(dp[w], dp[w - weights[i]] + values[i]);
9             }
10        }
11    }
12    return dp[capacity];
13 }

```

5.2 Proof of Correctness

Why 1D DP Works

The 1D DP approach works because:

1. For each capacity w , we consider all items.
2. When we update $dp[w]$, we're finding the maximum value for capacity w considering all items processed so far.
3. Since we can reuse items (unbounded), we can use $dp[w-\text{weights}[i]]$ which may already include item i .
4. This is different from 0-1 Knapsack, where we must process capacities in reverse to avoid reusing items.

5.3 Implementation Comparison

6 Detailed Example

Let's solve an example step by step to understand how the algorithm works in practice.

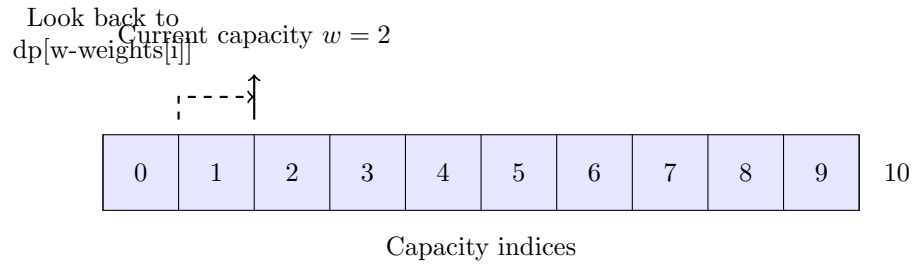


Figure 2: Visualization of 1D DP array update process

Aspect	2D DP	1D DP	Notes
Space Complexity	$O(n \times W)$	$O(W)$	1D is more memory efficient
Time Complexity	$O(n \times W)$	$O(n \times W)$	Same time complexity
Implementation	More natural to recurrence	More compact	1D is harder to derive intuitively
Tracing solution	Easier	Harder	Backtracking is easier in 2D

Table 2: Comparison between 2D and 1D DP approaches

Example Problem

Given:

- Weights: [1, 2, 3]
- Values: [10, 15, 40]
- Capacity: 6

Find: The maximum value that can be obtained without exceeding the capacity.

6.1 Tabulation Solution Walkthrough

We'll use the tabulation approach with a 2D array to solve this step by step.

1. **Initialize the DP table:** Create a table $dp[4][7]$ initialized with 0s ($n+1$ rows, capacity+1 columns).
2. **Fill the table using the recurrence relation:**

$$dp[i][w] = \max(dp[i-1][w], dp[i][w - weights[i-1]] + values[i-1]) \quad (7)$$

w/i	0	1	2	3
0	0	0	0	0
1	0	10	10	10
2	0	20	20	20
3	0	30	30	40
4	0	40	45	50
5	0	50	55	55
6	0	60	60	70

Table 3: DP Table for the example

6.2 Reconstructing the Optimal Solution

Finding the Items in the Optimal Solution

Once we have the DP table filled, we can reconstruct the items chosen:

1. Start from $dp[n][W] = 70$
2. Compare with $dp[n-1][W] = 60$ to see if item n was included
3. If included, subtract weight and value of item n , and continue
4. If not included, move to $dp[n-1][W]$ and continue

Reconstructing our example:

- $dp[3][6] = 70 \nless dp[2][6] = 60$, so item 3 is included
- Remaining capacity = $6 - 3 = 3$
- $dp[3][3] = 40 \nless dp[2][3] = 30$, so item 3 is not included again
- Instead, item 1 is included 3 times (check $dp[1][3] = 30$)

Final solution: $1 \times \text{item 3 (value 40)} + 3 \times \text{item 1 (value } 10 \times 3 = 30) = 70$

6.3 Verification

Let's verify that our solution is correct by checking all possible combinations:

- $6 \times \text{item 1: } 6 \times 10 = 60$
- $3 \times \text{item 2: } 3 \times 15 = 45$
- $2 \times \text{item 3: Not possible (exceeds capacity)}$
- $1 \times \text{item 3} + 3 \times \text{item 1: } 40 + 3 \times 10 = 70$
- $1 \times \text{item 3} + 1 \times \text{item 2} + 1 \times \text{item 1: } 40 + 15 + 10 = 65$
- $2 \times \text{item 2} + 2 \times \text{item 1: } 2 \times 15 + 2 \times 10 = 50$
- $3 \times \text{item 2: } 3 \times 15 = 45$

Our solution with a total value of 70 is indeed optimal.

7 Applications and Extensions

7.1 Practical Applications

The Unbounded Knapsack problem has numerous practical applications across various domains:

Real-World Applications

- **Rod Cutting Problem:** A company needs to cut metal rods into smaller pieces to maximize profit. Each length has a different market value.
- **Coin Change Problem:** A vending machine needs to dispense change using the minimum number of coins. Each coin denomination can be used multiple times.
- **Production Planning:** A factory must decide how many units of each product to manufacture to maximize profit given limited resources.
- **Budget Allocation:** A marketing team needs to allocate budget across different channels (TV, social media, print) to maximize reach or ROI.
- **Portfolio Optimization:** An investor wants to allocate capital across different assets to maximize returns while considering risk constraints.

7.2 Rod Cutting Problem

One of the most famous applications of Unbounded Knapsack is the Rod Cutting Problem:

Rod Cutting Problem

Given a rod of length n and a price list $price[i]$ for rod pieces of length i where $1 \leq i \leq n$, find the maximum value obtainable by cutting the rod into smaller pieces.

Listing 5: Rod Cutting Implementation

```

1 static int rodCutting(int[] prices, int length) {
2     int[] dp = new int[length + 1];
3
4     for (int i = 1; i <= length; i++) {
5         for (int j = 1; j <= i; j++) {
6             dp[i] = Math.max(dp[i], dp[i - j] + prices[j - 1]);
7         }
8     }
9     return dp[length];
10 }
```

7.3 Coin Change Problem

Another application is the Coin Change Problem:

Coin Change Problem

Given a set of coin denominations and a target amount, find the minimum number of coins needed to make the target amount.

Listing 6: Coin Change Implementation

```

1 static int coinChange(int[] coins, int amount) {
2     int[] dp = new int[amount + 1];
3     Arrays.fill(dp, amount + 1); // Initialize with a value larger than possible
4     dp[0] = 0;
5
6     for (int i = 1; i <= amount; i++) {
7         for (int coin : coins) {
8             if (coin <= i) {
9                 dp[i] = Math.min(dp[i], dp[i - coin] + 1);
10            }
11        }
12    }
13    return dp[amount] > amount ? -1 : dp[amount];
14 }
```

7.4 Extensions of the Problem

Several extensions of the Unbounded Knapsack problem exist, adding various constraints or variations:

- **Bounded Knapsack:** Each item has a specific limit on how many times it can be used.
- **Multiple Knapsacks:** Multiple knapsacks with different capacities are available.
- **Multi-dimensional Knapsack:** Items have multiple weight dimensions (e.g., weight and volume) with separate constraints.
- **Fractional Unbounded Knapsack:** Items can be taken in fractional amounts.
- **Stochastic Knapsack:** Weights or values are probabilistic rather than deterministic.

8 Key Insights and Best Practices

Key Insights

1. **Optimal Substructure:** The Unbounded Knapsack problem exhibits optimal substructure, making it suitable for dynamic programming.
2. **State Definition:** The state $dp[i][w]$ represents the maximum value achievable using the first i items with capacity w .
3. **Recurrence Relation:** The key insight is that after including an item, we can use it again (staying at the same item index).
4. **Space Optimization:** The 2D DP table can be optimized to a 1D array, reducing space complexity from $O(n \times W)$ to $O(W)$.
5. **Time Complexity:** The solution has a pseudo-polynomial time complexity of $O(n \times W)$, where n is the number of items and W is the capacity.

8.1 Implementation Best Practices

When implementing the Unbounded Knapsack algorithm, consider these best practices:

- **Use 1D DP Array:** For most applications, the 1D DP approach is sufficient and more memory-efficient.
- **Initialize Properly:** For maximization problems, initialize to 0; for minimization problems, initialize to a large value.
- **Handle Edge Cases:** Ensure proper handling of edge cases, such as when the capacity is 0 or no items are available.
- **Reconstruct Solution:** If needed, maintain additional information to reconstruct the actual items selected.
- **Optimize Item Order:** For some variants, processing items in a specific order can lead to more efficient solutions.

8.2 Common Pitfalls

Be aware of these common pitfalls when solving Unbounded Knapsack problems:

- **Confusing with 0-1 Knapsack:** Using $dp[i-1][w - \text{weights}[i-1]]$ instead of $dp[i][w - \text{weights}[i-1]]$.
- **Incorrect Base Cases:** Forgetting to initialize the DP table properly.
- **Integer Overflow:** For large inputs, integer overflow might occur.
- **Inefficient Solution Reconstruction:** Not efficiently tracking the selected items.
- **Assuming Greedy Works:** Unlike some problems, Unbounded Knapsack cannot be solved optimally with a greedy approach.

9 Conclusion

The Unbounded Knapsack problem represents an important extension of the classic 0-1 Knapsack problem by allowing unlimited use of items. This seemingly small change leads to significant differences in the algorithm design and implementation.

9.1 Summary of Key Concepts

- **Problem Definition:** Maximize value by selecting items (potentially multiple times) without exceeding capacity.
- **Recurrence Relation:** $OPT(i, w) = \max(OPT(i - 1, w), v_i + OPT(i, w - w_i))$
- **Solution Approaches:**
 - Recursive (brute force): Simple but exponential time complexity
 - Memoization (top-down DP): Efficiently computes only needed states
 - Tabulation (bottom-up DP): Systematically fills a table with no recursion
 - Space-optimized (1D DP): Reduces space complexity to $O(W)$
- **Key Difference from 0-1 Knapsack:** After including an item, we can reuse it (stay at the same item).
- **Applications:** Rod cutting, coin change, resource allocation, production planning, and more.

9.2 Future Directions

Research on the Unbounded Knapsack problem continues in several directions:

- **Approximation Algorithms:** For very large instances where exact solutions are impractical.
- **Parallel Implementations:** Leveraging parallel computing to solve larger instances.
- **Multi-objective Variants:** Considering multiple conflicting objectives simultaneously.
- **Online Variants:** Where items arrive one by one and decisions must be made immediately.
- **Integration with Machine Learning:** Using ML to predict parameters or guide the search for solutions.

9.3 Final Thoughts

Understanding both the 0-1 and Unbounded Knapsack problems provides a solid foundation for tackling many dynamic programming problems in optimization. The techniques learned here—identifying optimal substructure, formulating recurrence relations, and implementing efficient algorithms—are applicable to a wide range of problems in computer science and operations research.

As with many dynamic programming problems, the Unbounded Knapsack problem demonstrates how complex optimization challenges can be broken down into simpler subproblems whose solutions can be combined to solve the original problem efficiently.

Final Note

The key insight that distinguishes the Unbounded Knapsack problem is the ability to reuse items. This seemingly small difference changes the entire algorithm structure and solution space, highlighting how subtle variations in problem constraints can lead to significant changes in solution approaches.